

CafeConnect

Запуск · Тестирование · Сборка

Пошаговое руководство с нуля до готового APK. Читай с первой страницы каждый раз, когда нужно поднять окружение, протестировать на телефоне или собрать релизную сборку.



00 Содержание

01 Что нужно установить (один раз)

02 Скачать проект с GitHub

03 Настройка сети (телефон + компьютер)

04 Запустить бэкенд (Docker)

05 Запустить приложение на телефоне

06 Запустить Claude Code (работа с ИИ)

07 Сохранить изменения (Git)

08 Собрать релизный APK

09 Ошибки и как их чинить

10 Быстрые команды (шпаргалка)

Как читать этот документ

Шаги 01–02 — делаешь **один раз** при первой настройке.

Шаги 03–05 — делаешь **каждый раз** когда садишься работать.

Шаги 06–07 — делаешь когда **работаешь с кодом**.

Шаг 08 — делаешь когда нужно **собрать APK** для установки.

01 Что нужно установить

Делаешь один раз на своём компьютере. Потом не нужно повторять.

Flutter SDK

1 Скачай Flutter

Зайди на `flutter.dev/docs/get-started/install` и выбери свою ОС (Mac / Windows / Linux). Скачай архив и распакуй в удобную папку.

2 Добавь Flutter в PATH

Чтобы команда `flutter` работала в любом терминале. Инструкция на сайте `flutter.dev` — выбери свою ОС.

3 Проверь что всё работает

Открой терминал и выполни:

```
flutter doctor
```

Ты увидишь список — зелёные галочки = хорошо. Красные — нужно доустановить. Главное чтобы были галочки у

Flutter

и

Android toolchain

.

Docker Desktop

Скачай с `docker.com/products/docker-desktop`. Это нужно чтобы запускать бэкенд (Django). Установи и запусти приложение Docker Desktop — оно должно работать в фоне.

VS Code (редактор кода)

Скачай с `code.visualstudio.com`. После установки открой VS Code и установи расширение **Flutter** (нажми `Ctrl+Shift+X`, найди "Flutter", установи).

Git

На Mac и Linux обычно уже стоит. На Windows скачай с `git-scm.com` . Проверь: `git --version`

Node.js (для Claude Code)

Скачай с `nodejs.org` — версию LTS. Проверь: `node --version` — должен показать номер версии.

✓ Проверочный список — Шаг 01

- ☐ `flutter doctor` показывает зелёные галочки у Flutter и Android
- ☐ Docker Desktop запущен, иконка в трее видна
- ☐ VS Code открывается, расширение Flutter установлено
- ☐ `git --version` — показывает номер
- ☐ `node --version` — показывает номер

02 Скачать проект с GitHub

Делаешь один раз. Получаешь папку с проектом на своём компьютере.

1 Открой терминал и перейди в нужную папку

Выбери куда хочешь положить проект. Например, папка Documents:

```
# Mac / Linux
cd ~/Documents

# Windows
cd C:\Users\ТвоёИмя\Documents
```

2 Скачай репозиторий

```
git clone https://github.com/Dylachee/CafeConn.git
```

Появится папка `CafeConn`.

3 Перейди в папку и переключись на рабочую ветку

```
cd CafeConn
git checkout Uluk
```

4 Установи зависимости Flutter

```
flutter pub get
```

Скачает все библиотеки. Занимает 1–2 минуты.

5 Открой проект в VS Code

```
code .
```

03 Настройка сети

Делаешь каждый раз. Телефон должен «видеть» твой компьютер по сети.

Почему это нужно

Телефон не может подключиться к `localhost` — для него `localhost` это сам телефон. Нужно использовать реальный IP-адрес компьютера в локальной сети.

1 Убедись что телефон и компьютер в одной Wi-Fi сети

Оба устройства должны быть подключены к одному и тому же роутеру. Гостевые сети — отключи, они изолируют устройства.

Отключи VPN на компьютере и на телефоне — VPN мешает локальным соединениям.

2 Узнай IP-адрес компьютера

```
# Mac / Linux
ifconfig | grep "inet " | grep -v 127.0.0.1

# Windows (в командной строке cmd)
ipconfig
```

Ищи адрес вида `192.168.X.X` или `10.X.X.X` — это и есть твой локальный IP.

Запиши этот адрес

— он понадобится в следующих шагах.

⚠ Важно — IP меняется!

Каждый раз когда ты переподключаешься к Wi-Fi, IP может измениться. Если вдруг перестало работать — повтори шаг выше и проверь адрес.

04 Запустить бэкенд (Docker)

Делаешь каждый раз перед работой. Бэкенд — это «мозг» приложения, он хранит все данные.

1 Убедись что Docker Desktop запущен

Иконка Docker должна быть видна в трее (нижний правый угол на Windows, верхний правый на Mac). Если нет — открой Docker Desktop.

2 Перейди в папку бэкенда

В твоём проекте есть папка с Django. Обычно она называется `backend` или `server` :

```
cd ~/Documents/CafeConn/backend
```

3 Пропиши свой IP в настройках

Открой файл `.env` в папке бэкенда (создай если нет) и добавь строку с твоим IP:

```
# Замени на твой реальный IP из Шага 03
DJANGO_LAN_HOST=192.168.1.50
```

4 Запусти контейнеры

```
docker compose down
docker compose up -d
```

Первый запуск занимает 3–5 минут (скачивает образы). Последующие — несколько секунд.

5 Проверь что всё запустилось

```
docker compose ps
```

Должны быть строки со статусом `Up` для Django и Redis. Если видишь `Exit` — смотри раздел 09 «Ошибки».

6 Проверь через браузер

Открой на компьютере: `http://192.168.1.50:8000/admin` (свой IP). Должна появиться страница входа Django. Это значит бэкенд работает.

✓ **Бэкенд работает, если:**

- ☐ `docker compose ps` показывает все контейнеры со статусом Up
- ☐ Браузер открывает `http://ТВОЙ-IP:8000/admin`
- ☐ Redis тоже Up (нужен для WebSocket-ов)

05 Запустить приложение на телефоне

Делаешь каждый раз чтобы протестировать изменения на реальном устройстве.

Подготовка телефона (один раз)

1 Включи режим разработчика на Android

Настройки → О телефоне → Номер сборки. Нажми 7 раз подряд. Появится «Режим разработчика включён».

2 Включи отладку по USB

Настройки → Для разработчиков → Отладка по USB. Включи.

3 Подключи телефон к компьютеру кабелем

На телефоне появится запрос «Разрешить отладку по USB?» — нажми **ОК**

. Поставь галочку «Всегда разрешать с этого компьютера».

4 Проверь что телефон виден

```
flutter devices
```

Должен появиться твой телефон в списке.

Запуск приложения

1 Перейди в папку Flutter-приложения

```
cd ~/Documents/CafeConn
```

2 Запусти на телефоне с нужными параметрами

Замени значения на свои (IP, логин, пароль тестового пользователя):

```
flutter run \
  --dart-define=API_BASE_URL=http://192.168.1.50:8000 \
  --dart-define=API_USERNAME=test_user \
  --dart-define=API_PASSWORD=test_password
```

Первый запуск занимает 2–3 минуты (компиляция). Потом — несколько секунд.

Горячая перезагрузка (Hot Reload)

Пока `flutter run` работает, нажми `r` в терминале — приложение обновится за 1–2 секунды без перезапуска. Нажми `R` (большая) — полный перезапуск приложения. Нажми `q` — остановить.

✓ Приложение работает правильно, если:

- ☐ Видишь экраны SafeConnect, а не белый экран
- ☐ Нет надписи «Demo Mode» или «Offline»
- ☐ Данные реальные (те что в Django), а не демо
- ☐ Шрифт читаемый, цвета тёплые кремовые

06 Работа с Claude Code

Делаешь когда нужно изменить код — ИИ-агент сам правит файлы проекта.

1 Установи Claude Code (один раз)

```
npm install -g @anthropic-ai/claude-code
```

2 Перейди в папку проекта

```
cd ~/Documents/CafeConn
```

3 Запусти Claude Code

```
claude
```

Откроется интерактивный интерфейс в терминале.

4 Вставь промт и дай задание

Скопируй промт из файла `CAFECONNECT_CLAUDE_CODE_PROMPT_V2.md` и вставь в Claude Code. Потом скажи что нужно сделать.

5 Пока Claude Code работает — проверяй на телефоне

После каждого изменения нажми `r` в терминале где запущен `flutter run` — увидишь изменения сразу.

⚠ Правила безопасности при работе с Claude Code

Claude Code не должен делать `git commit` или `git push`. Это делаешь только ты сам после проверки. Если агент предлагает — откажись. Смотри Шаг 07.

07 Сохранить изменения (Git)

Делаешь после того как проверил изменения на телефоне и всё работает.

1 Проверь что анализатор кода не ругается

```
flutter analyze
```

Должно быть **0 errors**. Если есть ошибки — сначала исправь.

2 Посмотри что изменилось

```
git status
```

Покажет список изменённых файлов.

3 Добавь изменения и сохрани

```
# Добавить все изменённые файлы
git add .

# Сохранить с описанием что сделал
git commit -m "Описание того что изменил"

# Примеры хороших описаний:
# "Fix guest count stepper in table detail"
# "Add delivered toggle with strikethrough"
# "Phase 3: Tables screen pixel-perfect"
```

4 Отправь на GitHub

```
git push origin Uluk
```

Если что-то сломалось — откат

Если после изменений что-то перестало работать, можно вернуться к последнему сохранённому состоянию:

```
# Отменить все незакоммиченные изменения
git checkout .

# Вернуться к определённому коммиту (осторожно!)
git revert HEAD
```

08 Собрать релизный APK

Делаешь когда нужен APK для установки на телефон без компьютера.

1 Убедись что код без ошибок

```
flutter analyze
```

Должно быть 0 errors.

2 Собери APK

```
flutter build apk --release --split-per-abi \  
  --dart-define=API_BASE_URL=http://192.168.1.50:8000 \  
  --dart-define=API_USERNAME=test_user \  
  --dart-define=API_PASSWORD=test_password
```

Занимает 3–7 минут. `--split-per-abi` создаёт отдельные файлы для разных процессоров — они меньше по размеру.

3 Найди готовый APK

```
build/app/outputs/flutter-apk/
```

Там будет несколько файлов. Для большинства Android телефонов подходит `app-arm64-v8a-release.apk`.

4 Установи на телефон

Перекинь APK на телефон (через кабель, Telegram себе, или AirDrop). Открой файл — телефон попросит разрешить установку из неизвестных источников. Разреши и установи.

⚠ Важно для релизного APK

IP-адрес `192.168.1.50` вшит в APK при сборке. Если IP сервера изменится — нужно пересобирать APK с новым IP. Для стабильной работы лучше назначить серверу статический IP в настройках роутера.

09 Ошибки и как их чинить

Ошибка	Что это значит	Как починить
Connection refused	Приложение не может достучаться до бэкенда	1. Проверь что Docker запущен и все контейнеры Up. 2. Проверь что телефон и компьютер в одной сети. 3. Проверь IP — он мог измениться. 4. Отключи VPN.
DisallowedHost (в логах Docker)	Django не принимает запросы с твоего IP	Открой <code>.env</code> файл бэкенда, проверь что в <code>DJANGO_LAN_HOST</code> стоит твой текущий IP. Перезапусти Docker: <code>docker compose down</code> && <code>docker compose up -d</code>
WebSocket не подключается	Realtime соединение не работает	1. Убедись что Redis контейнер запущен (<code>docker compose ps</code>). 2. Проверь что токен авторизации не истёк. 3. Попробуй перезайти в приложение.
Приложение показывает Demo Mode	Подключение к бэкенду не удалось, откат на локальные данные	1. Проверь все параметры в команде <code>flutter run</code> — IP, логин, пароль. 2. Убедись что тестовый пользователь существует в Django. 3. Проверь бэкенд через браузер.
flutter analyze — ошибки	В коде есть синтаксические ошибки	Прочитай что написано в ошибке — там указан файл и строка. Открой файл в VS Code, он подчеркнёт красным где проблема. Если Claude Code сделал изменения — попроси его исправить.
flutter pub get — ошибки	Не удаётся скачать зависимости	Проверь интернет-соединение. Попробуй ещё раз. Если не помогает — попробуй <code>flutter clean</code> потом снова <code>flutter pub get</code> .
Шрифт в APK выглядит неправильно	Шрифты не подключены как ассеты	Убедись что файлы <code>.ttf</code> лежат в папке <code>assets/fonts/</code> и прописаны в <code>pubspec.yaml</code> . Не должно быть использования <code>GoogleFonts</code> в коде.
No devices found	Flutter не видит телефон	1. Проверь кабель. 2. Убедись что на телефоне разрешена отладка по USB. 3. Разблокируй телефон — иногда помогает. 4. Попробуй другой порт USB.

Смотреть логи Docker

Если что-то не так с бэкендом — смотри логи:

```
# Логи всех контейнеров
docker compose logs

# Логи только Django (последние 50 строк)
docker compose logs --tail=50 web

# Следить за логами в реальном времени
docker compose logs -f
```


10 Быстрые команды (шпаргалка)

Все нужные команды в одном месте. Скопируй нужную и вставь в терминал.

Каждый день (старт работы)

```
# 1. Перейти в проект
cd ~/Documents/CafeConn

# 2. Получить последние изменения с GitHub
git pull origin Uluk

# 3. Запустить бэкэнд
cd backend && docker compose up -d && cd ..

# 4. Запустить приложение на телефоне (замени IP и данные)
flutter run \
  --dart-define=API_BASE_URL=http://192.168.1.50:8000 \
  --dart-define=API_USERNAME=test_user \
  --dart-define=API_PASSWORD=test_password
```

Во время работы

```
# Горячая перезагрузка (нажать в терминале где запущен flutter run)
r      # быстро — обновляет UI
R      # медленнее — полный перезапуск
q      # остановить

# Проверить код на ошибки
flutter analyze

# Запустить Claude Code
claude
```

Сохранить работу

```
# Проверить что без ошибок
flutter analyze

# Сохранить изменения
git add .
git commit -m "Что сделал"
git push origin Uluk
```

Собрать APK

```
flutter build apk --release --split-per-abi \  
  --dart-define=API_BASE_URL=http://192.168.1.50:8000 \  
  --dart-define=API_USERNAME=test_user \  
  --dart-define=API_PASSWORD=test_password  
  
# Готовый APK:  
build/app/outputs/flutter-apk/app-arm64-v8a-release.apk
```

Docker

```
# Запустить бэкенд  
docker compose up -d  
  
# Остановить бэкенд  
docker compose down  
  
# Проверить статус контейнеров  
docker compose ps  
  
# Смотреть логи  
docker compose logs -f  
  
# Перезапустить если что-то не так  
docker compose down && docker compose up -d
```

Git — полезные команды

```
# Посмотреть что изменилось  
git status  
  
# Посмотреть историю коммитов  
git log --oneline -10  
  
# Отменить все незакоммиченные изменения (ОСТОРОЖНО!)  
git checkout .  
  
# Узнать на какой ветке ты сейчас  
git branch
```

Запиши свои данные

Мой IP

Логин test

Пароль test

Путь к проекту
